VIET NAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

Report

Week 5

Course: Applications of Computer Graphics

Supervisors:
Nguyễn Trọng Việt
Phạm Thanh Tùng
Võ Thế Hào

Student:
23127266 - Nguyễn Anh Thư

March 20, 2026

Acknowledgment

I would like to express sincere gratitude to Pham Thanh Tung for his guidance and support throughout the course.

Contents

Chapter I: Introduction **1**

Chapter II: Guidance **2**

1 Access 2

2 Local setup 2

3 Usage 2

Chapter III: Implementation **3**

1 Processing Flow 3

2 Interface Structure 3

3 Core Browser Functions 3

4 Frame Extraction 3

5 Edge Detection Pipeline 3

6 Real-Time Scheduling 4

Chapter IV: Limitations, Future Work, and Conclusion **5**

Reference **6**

Chapter I: Introduction

This project develops a lightweight web application using HTML5 `<video>` and Canvas for real-time edge detection. During playback, each displayed frame is processed locally in the browser using client-side JavaScript, and the resulting edge map is rendered alongside the original video.

Chapter II: Guidance

1. Access

- **GitHub repository:** https://github.com/ariesanhthu/homework5
- **GitHub Pages:** https://ariesanhthu.github.io/homework5/

2. Local setup

To run the application locally, install the required dependencies, start the Node.js server, and open the local host address in a web browser.

3. Usage

The user can either select a local video file or load the bundled sample video. During playback, the interface presents two synchronized views: the original video and the corresponding edge map generated in real time on the Canvas. The page also provides a threshold mode selector (*Manual* or *Adaptive*) and a slider for tuning the high-threshold behavior. Processing stops automatically when playback is paused or completed.

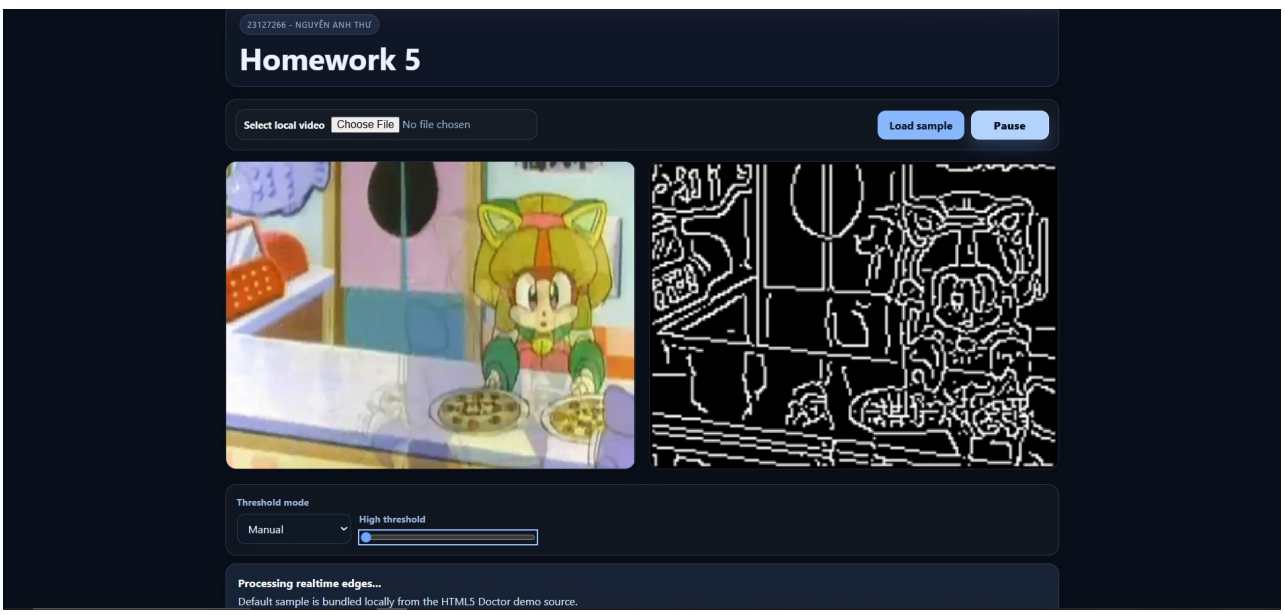

Figure II.1: Real-time edge output generated from the video stream.

Chapter III: Implementation

1. Processing Flow

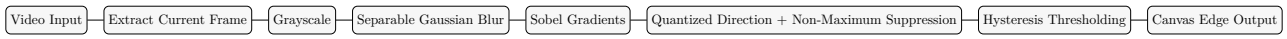

Figure III.1: Real-time frame processing pipeline.

2. Interface Structure

The interface contains a local file picker, a sample-loading button, a play/pause button, a `video` element for the input stream, and a visible Canvas for the edge output. It also includes a threshold mode selector, a threshold slider, and status text for reporting the current processing state. Internally, an offscreen Canvas is used for intermediate pixel-level computation.

3. Core Browser Functions

The processing workflow is controlled by video playback events and a frame-synchronized rendering loop. The main browser and Canvas APIs used are: `play()`, `pause()`, `addEventListener()`, `URL.createObjectURL()`, `getContext('2d')`, `drawImage()`, `getImageData()`, `putImageData()`, `clearRect()`, `requestAnimationFrame()`, and `cancelAnimationFrame()`.

4. Frame Extraction

The system does not process the raw video file directly. Instead, it processes the currently displayed frame. For each iteration, the current frame is copied from the `video` element to the internal processing Canvas using `drawImage()`, then converted into a pixel buffer through `getImageData()` for subsequent edge computation. Importantly, this internal Canvas runs at a reduced processing resolution controlled by `processingScale = 0.35`, after which the binary edge result is written back and upscaled onto the visible output Canvas.

5. Edge Detection Pipeline

The edge detector follows a compact Canny-style pipeline optimized for real-time execution:

1. **Grayscale conversion:** RGB values are converted into intensity values using an integer approximation of the standard luminance formula.

2. **Noise reduction:** a separable Gaussian blur with a 5-tap kernel is applied horizontally and vertically to suppress noise with low computational cost.

3. **Gradient estimation:** Sobel operators are used to compute horizontal and vertical gradients, from which gradient magnitude and a 4-direction quantized orientation are obtained.

4. **Non-maximum suppression:** local maxima are preserved along the gradient direction to produce thinner and more precise edges.

5. **Thresholding with hysteresis:** the system supports both a manual high threshold and an adaptive high threshold derived from the mean gradient magnitude. The low threshold is computed as a fixed

ratio of the high threshold. Strong edges are retained directly, while weak edges are preserved only if connected to strong responses in the 8-neighborhood.

6. Real-Time Scheduling

The processing loop is driven by `requestAnimationFrame()` to remain synchronized with browser rendering. Redundant processing is reduced by skipping frames whose `currentTime` has not changed enough since the previous processed frame. For efficiency, the implementation also reuses preallocated typed buffers for grayscale, blur, gradient, suppression, classification, and hysteresis traversal.

Chapter IV: Limitations, Future Work, and Conclusion

- Performance depends on video resolution and device capability.
- Edge quality is sensitive to threshold selection and blur configuration.
- The implementation prioritizes responsiveness and portability over the accuracy of specialized vision libraries.

Future improvements include moving computation to *OffscreenCanvas* or *Web Workers*, and extending the parameter control strategy with more adaptive threshold selection.

In conclusion, this project demonstrates that real-time edge detection can be achieved efficiently in a browser using only HTML5 Video, Canvas, and client-side JavaScript. The implementation combines frame extraction, grayscale conversion, Gaussian smoothing, Sobel gradients, non-maximum suppression, and hysteresis thresholding in a fully local processing pipeline.

Reference

[1] Application graphics: Web graphics environment. Week 7–8 lecture slides, 2025.